

ICS-SimLab: A Containerized Approach for Simulating Industrial Control Systems for Cyber Security Research

Jaxson Brown[†], Duc-Son Pham[†], Sie-Teng Soh[†], Foad Motalebi[‡], Sivaraman Eswaran[‡], and Mahathir Almashor[§]

[†] School of Electrical Engineering, Computing and Mathematical Sciences, Curtin University, Western Australia

[‡] Department of Electrical and Computer Engineering, Curtin University, Malaysia

[§] Commonwealth Scientific and Industrial Research Organisation (CSIRO) Energy, Perth, Western Australia

jokesene@outlook.com, dspham@ieee.org, s.soh@curtin.edu.au,

foad.m@curtin.edu.my, sivaraman.eswaran@curtin.edu.my, mahathir.almashor@csiro.au

Abstract—Industrial Control Systems (ICSs) are complex interconnected systems used to manage process control within industrial environments, such as chemical processing plants and water treatment facilities. As the modern industrial environment moves towards Internet-facing services, ICSs face an increased risk of attacks that necessitates ICS-specific Intrusion Detection Systems (IDS). The development of such IDS relies significantly on a simulated testbed as it is unrealistic and sometimes hazardous to utilize an operational control system. Whilst some testbeds have been proposed, they often use a limited selection of virtual ICS simulations to test and verify cyber security solutions. There is a lack of investigation done on developing systems that can efficiently simulate multiple ICS architectures.

We present ICS-SimLab, an end-to-end software suite that utilizes Docker containerization technology to create a highly configurable ICS simulation environment. This software framework enables researchers to rapidly build and customize different ICS environments, facilitating the development of security solutions across different systems that adhere to the Purdue Enterprise Reference Architecture. To demonstrate its capability, we present three virtual ICS simulations: a solar panel smart grid, a water bottle filling facility, and a system of intelligent electronic devices. Furthermore, we run cyber-attacks on these simulations and construct a dataset of recorded malicious and benign network traffic to be used for IDS development.

Index Terms—Industrial control systems, virtual testbed, simulation, dataset, intrusion detection systems

I. INTRODUCTION

Industrial Control Systems (ICSs) are complex, highly interconnected systems used to manage process control and automation across industrial environments. It is a general term encompassing Supervisory Control and Data Acquisition (SCADA) systems, Distributed Control Systems (DCS), as well as components such as Programmable Logic Controllers (PLC) and Remote Terminal Units (RTU) [1]. ICSs are found in facilities such as water treatment plants, transport infrastructure, power generation systems, etc. [2].

With the convergence of Information Technology (IT) and Operational Technology (OT), many ICS environments are increasingly exposed to cyber threats. This is especially prevalent with the use of Internet-facing services, which introduces

remote attack vectors [3]. As such, there is growing demand for effective cyber security solutions for ICS technology.

Historical examples of past ICS cyber attacks highlight the importance of implementing innovative security solutions. A commonly referenced example includes the Stuxnet attack. Stuxnet was a highly sophisticated malware that was designed to specifically target ICS environments [4], [5]. In 2010, Stuxnet was responsible for an attack on Iranian uranium enrichment facilities, which involved the physical destruction of centrifuges used in the enrichment process.

Considering the magnitude of past attacks, it's clear that cyber protection and solution development must be a priority for the modern ICS landscape. To be able to develop and evaluate such security solutions, researchers require testbeds that replicate real-world control systems.

Within this field, researchers often use previously built simulations from other works. This is common practice as it allows researchers to avoid hardware related issues and other technical problems involved with setting up a testbed [6]. Researchers investigating Machine Learning (ML)-based IDS development can also leverage existing datasets specifically designed for training. These datasets are typically produced by running cyber-attacks on ICS simulations whilst recording network activity. Through using these datasets, researchers can avoid setting up a simulation entirely [7], [8]. However, there are drawbacks to using existing simulations and datasets. There is a lack of flexibility in defining custom test conditions, as well as custom cyber-attacks [6]. There is also the major issue of bias within developed solutions.

We present ICS-SimLab, a software suite capable of simulating various types of ICS environments. This software utilizes Docker containerization technology to realize the different components of ICS environments. We also implement Hardware-in-the-Loop modules to replicate the physical processes involved in these control systems. Due to the efficiency of containers, ICS-SimLab can be run entirely on a single computer with relatively minimal computational requirements.

This paper is organized as follows. In Section II we investigate the literature on ICS simulation designs and analyse

their advantages and flaws. Section III explores the structure of ICS, including the different components and protocols used within ICS environments. Section IV describes the design of ICS-SimLab and describes three preconfigured simulations built with the framework. Section V highlights a practical application of the simulation software through investigating the effects of different cyber-attacks. In this section we also present several datasets that can be used for IDS developments. Finally, Section VI discusses areas of improvement for future work and concludes the results of this paper.

II. RELATED WORK

We review three categories of ICS testbeds as defined in [9]: physical, virtual and hybrid. We examine the strengths of the testbed designs and comment on any limitations.

A. Physical Testbed

Physical ICS simulations are built using real ICS devices, such as industry-grade Programmable Logic Controllers (PLCs) and Intelligent Electronic Devices (IEDs). They are often designed to be fully working, miniaturized control systems.

Teixeira et al. [10] have built an ICS testbed representing a simple water tank facility. Their testbed involves a physical water tank equipped with sensors, actuators and a PLC to control water levels within the tank.

Mathur and Tippenhauer [11] introduced a small-scale water purification ICS facility, designed specifically for cyber security research. It is named the SwaT testbed for “Secure Water Treatment”, and features a six-stage water purification process.

Morris et al. [12] developed a physical testbed consisting of seven miniaturized control systems, which incorporated both serial and Ethernet communications. The systems using serial communication include a water storage tank, a raised water system, a gas pipeline, a conveyor belt setup, and an industrial blower system. Two Ethernet-based systems were also developed: a steel rolling operation setup and a hybrid smart grid transmission control system.

Overall, physical ICS testbeds provide a valuable environment for comprehensive cyber security research. However, to replicate a physical testbed is very time consuming and often impractical for researchers [6]. Therefore, it's worth looking into alternative approaches such as virtual or hybrid simulations.

B. Virtual Simulations

Virtual ICS simulations are built fully on virtualization technology, such as virtual machines [13] or with Docker containers [14]. As a result, these simulations can typically run on a single computer, offering researchers a flexible and easily deployable testbed.

Dehlaghi-Ghadim et al. [6] developed a framework for building virtual ICS simulations based on an Object-Oriented Programming (OOP) approach using the Python programming language. Their system utilizes Docker containerization technology to replicate ICS components in a deployable environment. However, a prominent flaw within their framework is its applicability for creating custom ICS simulations.

The authors of [15] present a Graphical Realism Framework for Industrial Control Systems (GRFICS), which is a framework that utilizes the Unity game engine to visualize ICSs. They simulate the Tennessee Eastman Process (TEP), a widely recognized industrial process originally designed for evaluating process control and monitoring methods [16].

TinyICS [17] is an open-source ICS simulator built on NS-3 that models industrial communication using the Modbus protocol and provides Python bindings for ease of use by control system engineers. While TinyICS focuses on simulating process behavior and capturing communication traffic in customizable industrial scenarios, our ICS-SimLab platform extends beyond single-system emulation to support the rapid deployment of diverse, Docker-based ICS architectures aligned with the Purdue Enterprise Reference Architecture.

C. Hybrid Simulations

Physical testbeds and virtual testbeds both have their benefits and drawbacks. Many researchers utilize the benefits of both approaches through developing hybrid ICS testbeds. Such a testbed utilizes real industrial hardware in conjunction with simulated physical processes.

A hybrid simulation developed in [18] simulates the Tennessee Eastman Process, similarly to [15]. In this testbed, the TEP environment is simulated using software on a dedicated PC with I/O modules connected to real PLCs.

CPS-DERMS [19] introduced a sophisticated multi-co-simulation hardware-in-the-loop testbed that integrates OPAL-RT for grid-side simulations, NS-3 for large-scale cyber simulations, and a custom Python-based DERMS platform to study vulnerabilities in DER-DERMS communication. While their focus was on demonstrating novel Man-in-the-Middle (MITM) attack strategies within power distribution systems, including payload manipulation and stealth techniques, our work with ICS-SimLab is oriented toward simulating cyber-physical threats in industrial control systems (ICS) within industrial plant environments.

D. Limitations and Areas for Improvement

Most ICS testbeds in literature have been designed to simulate one specific type of control system. Little work is directed towards developing a generalized system capable of simulating a broad range of control systems architectures.

Furthermore, the current landscape of ICS testbeds offer researchers limited flexibility when it comes to developing custom components and extensions. In physical and hybrid setups, this limitation is understandable as hardware must be purchased and configured to match the testbed environment. However for virtual simulations, having a system that allows researchers to configure and customize their own testbeds would be of great value to the research community.

The current virtualization technologies can also be improved to increase the efficiency of simulation deployment. Most virtual testbeds utilize virtual machines to implement different ICS devices. Whilst they offer accurate encapsulation, running many of them at the same time has significant computational

cost. Containers are useful alternatives to virtual machines as they instead virtualize at the OS level, meaning that they share kernel resources and don't virtualize the OS itself.

III. ICS ARCHITECTURE OVERVIEW

As explored previously, the variety of ICS environments across industrial applications differ significantly. Despite these differences, there are several shared characteristics amongst all control systems. These characteristics typically include common industrial devices, communication protocols, and topology architectures [20].

A. Components

Common ICS components and modules are described in the following sections. These components can all be simulated within ICS-SimLab.

1) *Programmable Logic Controllers (PLC)*: PLCs are responsible for handling logic and switching operations [1]. They manage field devices such as sensors and actuators, and perform tasks related to process control and data acquisition.

2) *Human Machine Interfaces (HMI)*: HMIs act as centralized control points within an ICS [21]. They act as clients to PLCs, and are used to request data for visualization and send commands for manual control.

3) *Sensors and Actuators*: These components are field devices that monitor and handle physical interactions. Sensors are devices that monitor process measurements and actuators are responsible for physical movements in process control.

4) *Hardware-in-the-Loop (HIL) Modules*: HIL components are used to simulate physical processes through mathematical models. They are used for testing purposes and can also facilitate physical processes for ICS testbeds.

B. Modbus Protocol

Modbus is the most widely used communication protocol for ICS [22]. The Modbus protocol is an open-source protocol used for communication amongst ICS devices.

Modbus has a master/slave architecture. Modbus-compliant devices can either act as a master or as a slave: master devices send Modbus "requests" and slave devices reply with Modbus "responses" [23]. There are two primary variants of the protocol: RTU and TCP. There is another variant called ASCII which is not commonly used.

The Modbus-RTU frame consists of four fields: address, function code, data, and a Cyclic Redundancy Check (CRC) field. The address field identifies the device and the function code is used to specify a predefined function that dictates the purpose of a Modbus request. The data field is a variable-length field that contains parameters for Modbus requests or return values for Modbus responses. Finally the CRC field is for error checking purposes. Modbus-TCP is an encapsulation of the RTU variant into a TCP/IP packet. It facilitates the use of the Modbus protocol over networks operating on the TCP/IP stack [24].

C. Purdue Enterprise Reference Architecture

Industrial Control Systems tend to follow an architecture called the Purdue Enterprise Reference Architecture [20]. This model describes an ICS in a layered approach as shown in Figure 1. Each layer represents a different level of component interaction. ICS-SimLab follows the Purdue model when constructing the architecture of configured ICS simulations.

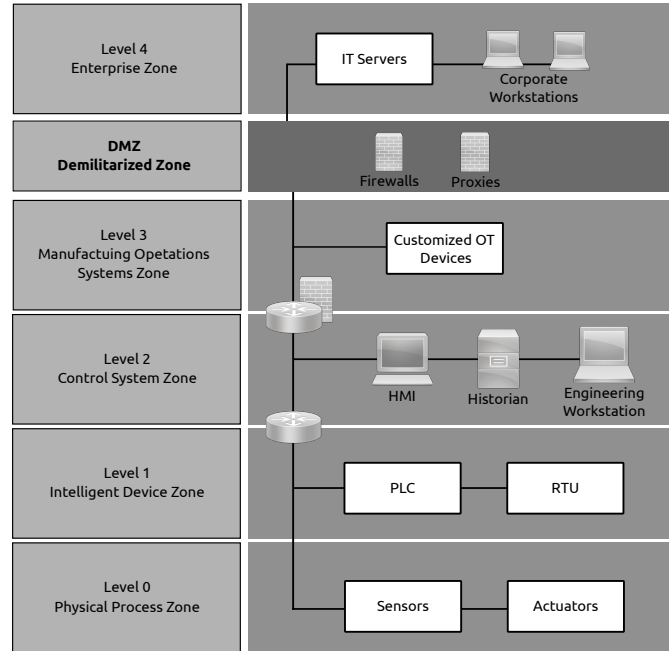


Fig. 1. Levels of the Purdue Enterprise Reference Architecture

IV. ICS-SIMLAB DESIGN AND IMPLEMENTATION

In this section we describe the design and implementation process of the ICS-SimLab simulation software. We elaborate on the reasoning behind key design choices and highlight innovations and features that can improve current ICS simulation practices. Additionally, we present three preconfigured simulations built using ICS-SimLab which can be readily used as immediate ICS testbeds.

A. Device Implementation

ICS-SimLab is capable of simulating all modules described in Table I. These components have been selected as they represent common devices responsible for core functionality in most ICS environments.

To achieve device virtualization and isolation, Docker containers have been used. The framework dynamically constructs containers for each component, enabling encapsulation and preventing direct communication between devices. To assist with running multiple containers in parallel, Docker Compose has been used.

Alternative virtualization methods, such as virtual machines, can introduce drawbacks related to computational cost and deployment complexity [25]. Unlike virtual machines, which

TABLE I
COMPONENTS SIMULATED BY ICS-SIMLAB

Component	Description
Human Machine Interfaces (HMI)	Displays collected data and enables manual control.
Programmable Logic Controllers (PLC)	Performs control logic for sensors and actuators.
Sensors	Reads environmental data from the HIL modules.
Actuators	Alters physical processes from the HIL modules.
Hardware-in-the-Loop (HIL) Modules	Simulates physical processes.

virtualize the entire physical computer, containers only virtualize the application environment. They share kernel resources whilst still providing process isolation. As a result, the container-based approach enables significantly more efficient simulation creation.

B. Network Emulation

To ensure accurate ICS communication simulation, Modbus-TCP and Modbus-RTU protocol variants have been implemented into ICS-SimLab.

To virtualize a TCP/IP network, Docker Compose has been used. Compose has in-built functionality that can create networks for containers to use. For Modbus-RTU, the `socket` utility has been used. `socket` has the ability to construct virtual serial ports, which ICS-SimLab then uses as a means for Modbus-RTU communication amongst containers.

Sensors and actuators require a means of communicating with the physical environment, which in ICS-SimLab simulates by the HIL modules. Similar to the hardware-aware layer presented in [6], ICS-SimLab uses a shared SQLite database among sensors, actuators, and HIL modules. Sensors read from the database, while actuators write to it. The physical values themselves are defined within a JSON configuration file, as described in Section IV-C. The HIL modules modify them based on customized logic implemented in Python.

C. Configuration System

ICS-SimLab uses a JSON configuration file to define all components of a simulation. Only one JSON file is needed per ICS type. For example, one file may define the components of a water storage tank system, while another specifies the layout of a gas pipeline simulation. Users can create their own JSON files to specify the components and functionalities they wish to incorporate. ICS-SimLab will then parse the JSON file and deploy a simulation based on the provided configuration.

The JSON file includes parent-level properties that correspond to the different devices defined in Table I. Within these properties, further configurations are required to define how the device works. Table II describes all of the key device-related JSON properties that can be defined.

Python has been used to develop the core configuration software of ICS-SimLab, including the functionality involved with booting up the various containers using the JSON file.

TABLE II
KEY PROPERTY CONFIGURATIONS IN THE JSON FILE

Attribute	Description	Components
name	Custom name for the component.	All components
network	IP and relevant Docker network interface.	All components
inbound_connections	Modbus slaves/server configurations.	PLC, sensors, actuators
outbound_connections	Modbus master/client configurations.	HMI, PLC
registers	Defines used Modbus register addresses.	HMI, PLC, sensors, actuators
monitors	Defines inter-device reading communication.	HMI, PLC
controllers	Defines inter-device writing communication.	HMI, PLC
logic	Specifies a Python file for custom logic implementation.	PLC, HIL
physical_values	Specifies environmental values.	HIL

The `pymodbus` Python library has been utilized to implement all Modbus communication. The containers themselves all run a Python service script with different logic depending on what device the container is simulating. For the components that depend on some predefined logic, such as the PLCs, additional Python scripts are created to provide a means of custom logic implementation, which are referenced in the JSON configuration file.

D. Preconfigured Simulations

To demonstrate the effectiveness of ICS-SimLab, three ICS simulations have been preconfigured. The first simulates a solar panel system, featuring a single HIL module that emulates solar power generation and a transfer switch that alternates input power between mains and solar. A PLC controls the switch based on the solar generation level, switching to mains if power drops below a set threshold, and back to solar otherwise. A HMI device is included for data acquisition across components. Figure 2 shows this simulation setup.

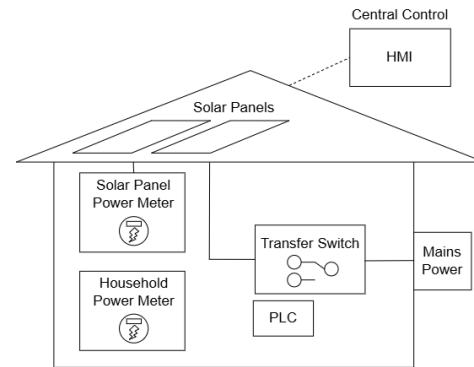


Fig. 2. Solar power setup simulation

The second simulation models a water bottle filling facility, shown in Figure 3, and is replicated from the testbed in [6].

It is divided into two sections, each controlled by a separate PLC. The first PLC manages water input and output for a tank, whose output valve leads to a conveyor belt that moves empty bottles beneath it. The second PLC controls the conveyor and signals the first PLC when a bottle is in position for filling.

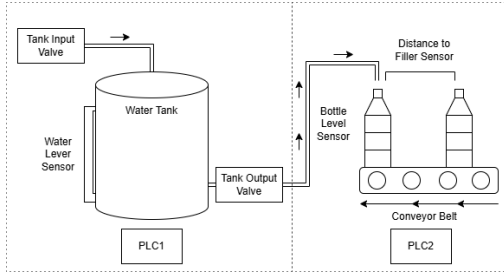


Fig. 3. Water bottle filling facility, inspired by the configuration in [6].

The third simulation models an Intelligent Electronic Device (IED) setup, inspired by the testbed in [14], which virtualizes an electrical substation. A simulated transformer performs random tap changes to vary its output voltage, either autonomously or via a HMI. If the voltage falls outside an acceptable range, a breaker trips until it stabilizes. All control logic in this simulation is handled by a PLC.

The source code for ICS-SimLab is publicly available in an online GitHub repository, along with the preconfigured simulation environments.¹

V. ICS-SIMLAB CYBER SECURITY APPLICATIONS

This section outlines the applications of ICS-SimLab in the context of cyber security research. Various ICS-specific cyber attacks are executed on the three preconfigured simulations, which are defined in Section IV-C, to investigate the effects on these attacks. A network data collection system is introduced to capture both benign and malicious activity. Finally, a data labeling system is used to generate datasets that can support the development of Intrusion Detection Systems (IDS) using Machine Learning (ML) techniques.

A. Cyber-Attacks and Vulnerabilities

ICS environments are vulnerable to attacks unique to their architecture, especially when compared to other systems such as traditional IT systems [1], [26]. Morris [27] presents a number of ICS cyber attacks. Table III displays all the attacks selected from [27] that have been used on the simulations generated with ICS-SimLab. These attacks are specific to ICS environments using the Modbus protocol, and are general enough to be able to be used on any control system type.

These attacks have a clear and expected effect on each of the simulations. As shown in Table III, reconnaissance attacks have the purpose of gathering device and network information. Response and measurement injection attacks involve crafting malicious Modbus packets for data manipulation. Command injection attacks abuse unsafe standard Modbus function codes

TABLE III
IMPLEMENTED CYBER ATTACKS BASED ON [27]

Attack Name	Classification	Description
Address Scan	Reconnaissance	An address scan to determine Modbus-compliant devices.
Function Code Scan	Reconnaissance	Scans for valid Modbus function codes.
Device Identification	Reconnaissance	Sends a Modbus request with function code 0x2B to return device-specific information.
Naive Sensor Read	Response and Measurement Injection	Scans over all Modbus registers and relays and attempts to find used addresses.
Sporadic Sensor Measurement Injection	Response and Measurement Injection	Writes random values to coils/holding registers.
Force Listen Mode	Command Injection	Forces a device into Force Listen Mode using Modbus.
Restart Communications	Command Injection	Forces a device restart using Modbus.
Data Flood Attack	Denial of Service	Floods the network with Modbus packets.
Connection Flood Attack	Denial of Service	Floods the network with TCP connection requests.

to issue unauthorized commands. Finally, denial of service attacks aim to overwhelm the system with excessive network packets to mask genuine network activity.

B. Dataset Generation

An attacking module was created with Python that would run random attacks from those listed in Table III. This module was deployed across all of the predefined ICS simulations, with Wireshark being used to capture the resulting network activity. After 30 minutes of simulating both benign and malicious network activity, the captured traffic data was exported to PCAP files for further processing.

TABLE IV
FEATURES OF THE GENERATED DATASET.

Feature	Description
time	Time of packet creation
src_mac	Source MAC address
dest_mac	Destination MAC address
src_ip	Source IP address
dest_ip	Destination IP address
protocol	The protocol of the packet.
length	Length of the Modbus Application Data Unit (ADU)
unit_id	Modbus unit ID
func_code	Modbus function code
data	The Modbus data field
attack_specific	The specific attack used (for malicious packets)
attack_category	Category of the attack (for malicious packets)
attack_binary	Defines if a packet is benign (0) or malicious (1)

A custom data processing script was used to extract relevant features from the PCAP files and construct a CSV-formatted dataset. Each packet was labeled as either malicious or benign, along with its corresponding attack category. Table IV

¹<https://github.com/JaxsonBrownie/ICS-SimLab>

describes all the features extracted for the dataset. The PCAP files and generated CSV files can be found in the GitHub repository for the source code for ICS-SimLab, along with the custom data processing script.

Since ICS-SimLab can simulate various ICS environments, it enables the generation of multiple datasets representing different architectures under attack. These datasets can be used collectively to develop a generalized IDS capable of detecting malicious activity across various systems. At the least, having access to multiple ICS environments allows researchers to evaluate the robustness of certain ML techniques against varying network behavior. Additionally, training across multiple datasets introduces the effect of data overfitting, which while a challenging issue to face, can ultimately support the development of more resilient IDS models by exposing their generalization limits.

VI. CONCLUSION AND FUTURE WORKS

This paper presented ICS-SimLab, a framework for generating ICS simulations using customizable configurations. ICS-SimLab uses containerization to enable full simulations to run efficiently on a single host and a JSON-based configuration system to allow researchers to build custom ICS testbeds.

To demonstrate the capabilities of ICS-SimLab, three pre-configured simulations have been developed. ICS-specific cyber attacks were designed and executed on all three simulations, and their effects were observed. Datasets were generated to represent network activity during both normal operations and attack scenarios, which can support the development of network-based Intrusion Detection Systems.

For future work, we aim to develop a real-time IDS software capable of detecting ICS-specific attacks. ICS-SimLab can facilitate this development through providing various ICS simulation environments for constructing and validating the IDS system. The datasets generated in this work can be used to train Machine Learning models used for building the IDS.

To enhance the ICS-SimLab framework, we aim to explore the use of more accurate tools for virtualizing specific components. Currently, Python is used to simulate physical processes. However, more specialized software such as Matlab or Simulink may offer improved accuracy [13].

ACKNOWLEDGMENT

This work is supported by a cross-campus cyber security seed grant from Curtin University.

REFERENCES

- [1] Z. Drias, A. Serhrouchni, and O. Vogel, "Analysis of cyber security for industrial control systems," in *2015 international conference on cyber security of smart cities, industrial control system and communications (ssic)*, pp. 1–8, IEEE, 2015.
- [2] U. P. D. Ani, J. M. Watson, B. Green, B. Craggs, and J. R. Nurse, "Design considerations for building credible security testbeds: Perspectives from industrial control system use cases," *Journal of Cyber Security Technology*, vol. 5, no. 2, pp. 71–119, 2021.
- [3] Y. Hu, A. Yang, H. Li, Y. Sun, and L. Sun, "A survey of intrusion detection on industrial control systems," *International Journal of Distributed Sensor Networks*, vol. 14, no. 8, p. 1550147718794615, 2018.
- [4] N. Falliere, L. O. Murchu, E. Chien, et al., "W32. stuxnet dossier," *White paper, symantec corp., security response*, vol. 5, no. 6, p. 29, 2011.
- [5] T. M. Chen and S. Abu-Nimeh, "Lessons from stuxnet," *Computer*, vol. 44, no. 4, pp. 91–93, 2011.
- [6] A. Dehlaghi-Ghadim, A. Balador, M. H. Moghadam, H. Hansson, and M. Conti, "Ic3sim—a framework for building industrial control systems security testbeds," *Computers in Industry*, vol. 148, p. 103906, 2023.
- [7] S. Mubarak, M. H. Habaebi, M. R. Islam, A. Balla, M. Tahir, A. Elsheikh, and F. Suliman, "Industrial datasets with ics testbed and attack detection using machine learning techniques," *Intell. Autom. Soft Comput*, vol. 31, no. 3, pp. 1345–1360, 2022.
- [8] T. Morris and W. Gao, "Industrial control system traffic data sets for intrusion detection research," in *Critical Infrastructure Protection VIII: 8th IFIP WG 11.10 International Conference, ICCIP 2014, Arlington, VA, USA, March 17-19, 2014, Revised Selected Papers 8*, pp. 65–78, Springer, 2014.
- [9] M. Conti, D. Donadel, and F. Turrin, "A survey on industrial control system testbeds and datasets for security research," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 4, pp. 2248–2294, 2021.
- [10] M. A. Teixeira, T. Salman, M. Zolanvari, R. Jain, N. Meskin, and M. Samaka, "Scada system testbed for cybersecurity research using machine learning approach," *Future Internet*, vol. 10, no. 8, p. 76, 2018.
- [11] A. P. Mathur and N. O. Tippenhauer, "Swat: A water treatment testbed for research and training on ics security," in *2016 international workshop on cyber-physical systems for smart water networks (CySWater)*, pp. 31–36, IEEE, 2016.
- [12] T. Morris, A. Srivastava, B. Reaves, W. Gao, K. Pavurapu, and R. Reddi, "A control system testbed to validate critical infrastructure protection concepts," *International Journal of Critical Infrastructure Protection*, vol. 4, no. 2, pp. 88–103, 2011.
- [13] T. Alves, R. Das, A. Werth, and T. Morris, "Virtualization of scada testbeds for cybersecurity research: A modular approach," *Computers & Security*, vol. 77, pp. 531–546, 2018.
- [14] K. Boakye-Boateng, A. A. Ghorbani, and A. H. Lashkari, "Securing substations with trust, risk posture, and multi-agent systems: A comprehensive approach," in *2023 20th Annual International Conference on Privacy, Security and Trust (PST)*, pp. 1–12, IEEE, 2023.
- [15] D. Formby, M. Rad, and R. Beyah, "Lowering the barriers to industrial control system security with {GRFICS}," in *2018 USENIX Workshop on Advances in Security Education (ASE 18)*, 2018.
- [16] A. Bathelt, N. L. Ricker, and M. Jelali, "Revision of the tennessee eastman process model," *IFAC-PapersOnLine*, vol. 48, no. 8, pp. 309–314, 2015.
- [17] A. Candia and C. Cappel, "Tinyics: An industrial control system simulator based on ns-3," in *Proc. 43rd International Conference of the Chilean Computer Science Society (SCCC)*, pp. 1–8, IEEE, 2024.
- [18] M. Noorzadeh, M. Shakerpour, N. Meskin, D. Unal, and K. Khorasani, "A cyber-security methodology for a cyber-physical industrial control system testbed," *IEEE Access*, vol. 9, pp. 16239–16253, 2021.
- [19] S. C. V. Kondu, G. Ravikumar, and S. N. Mohan, "Cps-derms: Cyber-physical security and impact analysis of derms against mitm attacks," in *Proc. IEEE Green Technologies Conference (GreenTech)*, pp. 1–5, IEEE, 2025.
- [20] T. J. Williams, "The purdue enterprise reference architecture," *Computers in industry*, vol. 24, no. 2-3, pp. 141–158, 1994.
- [21] V. M. Iguire, S. A. Laughter, and R. D. Williams, "Security issues in scada networks," *computers & security*, vol. 25, no. 7, pp. 498–506, 2006.
- [22] B. Chen, N. Pattanaik, A. Goulart, K. L. Butler-purpy, and D. Kundur, "Implementing attacks for modbus/tcp protocol in a real-time cyber physical system test bed," in *2015 IEEE International Workshop Technical Committee on Communications Quality and Reliability (CQR)*, pp. 1–6, 2015.
- [23] B. Dutertre, "Formal modeling and analysis of the modbus protocol," in *International Conference on Critical Infrastructure Protection*, pp. 189–204, Springer, 2007.
- [24] A. Swales et al., "Open modbus/tcp specification," *Schneider Electric*, vol. 29, no. 3, p. 19, 1999.
- [25] I. B. de Brito and R. T. de Sousa Jr, "Development of an open-source testbed based on the modbus protocol for cybersecurity analysis of nuclear power plants," *Applied Sciences*, vol. 12, no. 15, p. 7942, 2022.
- [26] K. Stouffer, J. Falco, K. Scarfone, et al., "Guide to industrial control systems (ics) security," *NIST special publication*, vol. 800, no. 82, pp. 16–16, 2011.
- [27] T. H. Morris and W. Gao, "Industrial control system cyber attacks," in *1st International Symposium for ICS & SCADA Cyber Security Research 2013 (ICS-CSR 2013)*, BCS Learning & Development, 2013.